

AFP Project

A Complete Onboarding Guide for New Contributors

Prepared for the Experiment Machine Operator

DGX Spark GB10 | 121 GB Unified Memory | ARM64 CUDA 13.0

July 23, 2026

1 In 30 Seconds: What Is This Project?

We take a 1.3-billion-parameter language model (Pythia), fine-tune it on two different domains (code reasoning and medical reasoning), and measure two things:

1. **How far do the weights move?** (Weight Divergence, ΔW)
2. **Can you draw a straight line between them without crossing a loss barrier?** (Linear Mode Connectivity, LMC)

Core finding: Training stability, not domain difference, determines the barrier height. Medical training is $3\times$ less stable than code training.

The paper is titled: “*Training Stability, Not Domain Difference, Determines Linear Mode Connectivity.*” Current version: v19 (20 pages, 4 domains, 2 model families, theory section).

2 The Hardware: Your Machine

You’re working on a **NVIDIA DGX Spark GB10**. Its key property:

- **121 GB unified memory** — CPU and GPU *share* the same physical RAM. There’s no separate VRAM.
- **ARM64 CPU** — This means `torch.compile()` does NOT work (Triton is x86-only).
- **CUDA 13.0** on Blackwell architecture (compute capability 12.1).
- Training speed: Pythia-1.4B full-FT at batch=128, seq_len=256: ≈ 5.5 s/step (≈ 42 min per epoch).

Never run `ipcrm` or `clear /dev/shm`! The DGX’s unified memory is shared with Tailscale’s runtime state. Clearing it will permanently destroy remote access — you’ll need physical access to the machine to fix it.

3 Project Map: Where Everything Lives

Directory / File	Type	What's Inside
CLAUDE.md	Instruction	Rules the AI must follow; core data table; “DO NOT” list
README.md	Public	GitHub landing page; paper title and abstract-level summary
paper.pdf	Deliverable	Compiled 20-page paper (authoritative)
docs/internal/paper.tex	Source	LaTeX source v19 — the authoritative paper text
docs/internal/PAPER.md	Mirror	Markdown copy of the paper (convenience; LaTeX is authoritative)
docs/internal/APPENDIX.md	Data	Numerical appendix tables
docs/internal/ENGINEERING.md	Manual	23 documented bugs with root causes + fixes + lessons
docs/internal/ICLR_SPRINT_PLAN.md	Roadmap	3-phase experiment plan, timeline, GPU budget
docs/internal/EXPERT_PANEL_FINDINGS.md	Review	6-expert adversarial review (v4 era, historical)
docs/internal/DATA_INVENTORY.md	Catalog	Complete list of all 42+ models and 110+ scans
docs/reports/FIGURE_SPEC_V2.md	Spec	Figure specifications — exact data, colors, layout
docs/reports/figure_data.json	Data	Pythia primary results (Figs 1–3)
docs/reports/fig*.pdf	Output	Vector figures for the paper
scripts/train_agent.py	Code	Full-FT training with drive-putt LR schedule
scripts/lmc_barrier_scan.py	Code	11-point LMC barrier measurement
scripts/lmc_3pt_scan.py	Code	Fast 3-point LMC scan (used for OPT)
scripts/gptneo_pipeline.sh	Pipeline	GPT-Neo full workflow: tokenize + train + LMC
scripts/generate_figures.py	Viz	Generate Fig 1–3 from <code>figure_data.json</code>
scripts/generate_fig4.py	Viz	Generate Fig 4 (cross-architecture)
experiments/trained_models/	Data	Pythia-1.4B checkpoints (288 GB, 240 .pt files)
experiments/trained_models_opt/	Data	OPT-1.3B checkpoints (184 GB)
experiments/phase0_ivn/results/	Data	122 LMC barrier JSONs — primary data
experiments/phase0_opt/results/	Data	OPT replication results
data/versaprm/	Data	Tokenized training data (2.1 GB)

4 The Core Numbers

Condition	ΔW	Code Barrier	Medical Barrier
Standard divergence	1.4%	0.053 ± 0.011	0.051 ± 0.013
High divergence	8.0%	0.118 ± 0.031	0.228 ± 0.102
Within-domain (code)	—	0.048	—
Within-domain (medical)	—	—	0.147
Noise floor (identical)	—	≈ 0.000	≈ 0.000
Noise floor (random init)	—	0.033 ± 0.001	0.150 ± 0.019

Table 1: Core results table. Source: CLAUDE.md and paper §4.2–4.5.

What These Numbers Mean

- 1. Standard FT: LMC holds cleanly.** Barrier ≈ 0.05 is small. Models fine-tuned on code and medical from the same pretrained checkpoint remain in the same loss basin. You can linearly interpolate between them without a big loss penalty.
- 2. The Medical Anomaly.** Medical within-domain barrier (0.147) is $3\times$ the cross-domain barrier (0.051). Two models trained on the *same* medical data differ *more* in the loss landscape than a code model and a medical model. Training on medical data is inherently less stable.
- 3. Code is Remarkably Stable.** Code within-domain $\approx$ cross-domain ($0.048 \approx 0.053$). Domain difference contributes essentially nothing for code.
- 4. Barrier $\neq$ Weight Distance.** Gaussian noise at $\Delta W = 8\%$ produces barrier 0.014. Training at $\Delta W = 1.5\%$ produces barrier 0.053 — $9\times$ larger. It’s the *direction* of weight change, not the magnitude.

5 The Experiments

Core Measurements (Pythia-1.4B, 1 epoch)

- 1. Weight Divergence:** For each model pair, compute $\Delta W_i = \|\theta_i - \theta_{\text{base}}\|_2 / \|\theta_{\text{base}}\|_2$ for each layer, then average.
- 2. LMC Barrier Scan:** 11-point linear interpolation $\theta(\alpha) = (1 - \alpha)\theta_{\text{code}} + \alpha\theta_{\text{medical}}$, $\alpha \in \{0, 0.1, \dots, 1.0\}$. Measure BCE loss on 2,000 test samples per domain. Compute: barrier = $\max_{\alpha} \mathcal{L}(\theta(\alpha)) - (\mathcal{L}(\theta(0)) + \mathcal{L}(\theta(1))) / 2$.
- 3. Within-Domain Baseline:** Same-domain, different-seed pairs. Is the barrier from domain difference or from seed variance?
- 4. Seed-Pair Compatibility:** All $C(5, 2) = 10$ pairs of high-divergence medical models. Found $15\times$ range (0.072–1.213) — **pair-specific, not model-specific**.

Mechanistic Experiments

- 5. Training Trajectory:** Checkpoints every 40 steps. Code shows inverted-U (peak at step 200, then declines). Medical shows monotonic growth (plateaus at ≈ 0.21).

6. **Gaussian Perturbation:** Add isotropic noise at 5 ΔW levels. Near-zero barriers even at 8%. Proves barriers come from *structured* weight changes.
7. **Layer-Selective Interpolation:** Merge only subsets of layers. 75% of barrier lives in layers 0–7. Late layers (16–23): near-zero penalty.
8. **Merge Quality Benchmark:** Pure weight averaging beats TIES-Merging and Task Arithmetic. Accuracy barriers are much smaller than loss barriers.

Cross-Architecture Replication (OPT-1.3B)

OPT-1.3B (different architecture, corpus, tokenizer) confirms the medical > code within-domain pattern (ratio: $3.6\times$ vs. Pythia’s $3.7\times$). Absolute barriers are $4\text{--}8\times$ larger, but the relative ordering is preserved.

GPT-Neo-1.3B

Status: Pending. Training scripts exist (`gptneo_pipeline.sh`) but results are not yet reported.

6 How to Run Things

Environment Setup

```
export HF_ENDPOINT=https://hf-mirror.com
export HF_DATASETS_OFFLINE=1
export PYTHONUNBUFFERED=1
cd /home/jiayu/AFP
VENV=/home/jiayu/AFP/venv/bin/python3
```

Train a Single Model (≈ 42 min)

```
$VENV -u scripts/train_agent.py \
  --domain code --lr 1e-4 \
  --model-id EleutherAI/pythia-1.4b
```

Output goes to `experiments/trained_models/code/`.

Run an LMC Barrier Scan (≈ 15 min)

```
$VENV -u scripts/lmc_barrier_scan.py
```

Batch Pipeline ($\approx 2\text{--}3$ GPU-hours)

```
bash scripts/phase1_batch.sh      # LR sweeps + within-domain
bash scripts/final_batch.sh      # Merge benchmark + OPT trajectory
bash scripts/theory_experiments.sh # Label noise + extended putt + Hessian
```

Quick Status Check

```
bash scripts/monitor.sh
```

7 The 10 Iron Rules

These rules exist because each one was violated at least once, causing real damage.

1. **Use nohup** — All batch experiments must launch with `nohup ...&`.
2. **Redirect output** — `stdout/stderr` go to timestamped log files in `experiments/`.
3. **Zero stdin** — No `input()` or `sys.stdin.read()` in batch scripts.
4. **HF_DATASETS_OFFLINE=1** — Zero network during training/eval. Always.
5. **No set -e** — One experiment crashing must NOT kill the entire batch.
6. **Fail gracefully** — Use `|| { log "ERROR"; return 1; }`, never `exit 1`.
7. **Skip if done** — Check `[ -f "$out" ] && continue` for idempotent re-runs.
8. **Verify $\Delta W > 0.1\%$** — After every training run. Bug 22: base model was saved as “trained” model with zero weight change, invalidating ALL early LMC experiments.
9. **Verify process alive** — `sleep 3 && ps -p $PID` after launch.
10. **Log crashes** — Structured failure logs into `crashes/` directory.

Rule #8 is the most important. Bug 22 cost weeks of work. After every training: compute ΔW vs. the base checkpoint. If $\Delta W < 0.1\%$, the training failed silently and the output is garbage.

8 The “DO NOT” List

- **Do NOT mention LR in conclusions or titles.** LR is a knob for creating divergence, not a scientific contribution.
- **Do NOT mention AFP / IVN / gate.** That research direction is abandoned. `VISION.md` is historical archive only.
- **Do NOT chase an “LMC broke” narrative.** The experiments showed it didn’t break. The story is about *stability asymmetry*, not failure.
- **Do NOT use symlinks for model directories.** Every model lives in its own `{domain}_lr{lr}_s{seed}/` directory.
- **Do NOT trust caches blindly.** The cache filename includes `MAX_LEN`. If you change the config, delete the old cache.
- **Do NOT use `return_tensors='pt'` in tokenization loops.** It creates one tensor object per sample — OOM on large datasets.
- **Do NOT store token IDs as Python `list[int]`.** Python ints are $3.5\times$ larger than tensor `int64`.

9 The 23 Known Bugs

Read docs/internal/ENGINEERING.md for full details. Here are the 5 most important:

1. **Bug 11: .item() in inner loop $\rightarrow$ 10,416 GPU syncs.** MAS importance computation took 5+ hours instead of 1 minute. Fix: accumulate on GPU, sync once.
2. **Bug 12: MAS ran on CPU.** `to_device()` was called after the computation, not before. Fix: move model to GPU before any computation.
3. **Bug 21: ΔW not verified after training.** Base model was saved as “trained” with zero weight change. Fix: mandatory post-training ΔW check.
4. **Bug 22: `code_e1` was the untrained base model.** ALL early LMC experiments invalidated. The “code” model was just Pythia-1.4B with zero fine-tuning.
5. **Bug 23: Multi-seed training silently overwrites.** Output paths lacked seed differentiation. Running 3 seeds produced 1 model (the last one).

10 The Paper Structure

Section	Content
Abstract	Training stability, not domain difference, drives barrier height
§1 Intro	Why this matters for model merging and understanding fine-tuning
§2 Related Work	LMC, model merging, fine-tuning analysis — 24 citations
§3 Method	Pythia-1.4B full-FT, 11-point LMC, drive-putt LR schedule
§4.1 Weight Divergence	Per-block ΔW , $r = 0.995$ across domains
§4.2 LMC Barriers	Standard vs. high divergence, catastrophic intermediate spike
§4.3 Within-Domain	4-domain stability spectrum; seed-pair compatibility ($C(5,2)=10$)
§4.4 Asymmetry	Medical FT transfers positively to code, but not vice versa
§4.5 Noise Floor	Identical ≈ 0 , Random ref ≈ 0.150
§4.6 Trajectory	Code inverted-U vs. medical monotonic
§4.7 Gaussian	Structured $9\times$ unstructured displacement
§4.8 Layer-Selective	75% in layers 0–7, 0% in layers 16–23
§4.9 OPT	Cross-architecture: med/code ratio $3.6\times$ (OPT) vs. $3.7\times$ (Pythia)
§5 Theory	Hessian-aligned SGD diffusion; Peclet number; basin statistics
§6 Discussion	Merge quality, implications for continual learning
§7 Limitations	Permutation verified (neuron corr 0.984), single family addressed, convergence confound

11 The Figures

Fig	Panels	Status
-----	--------	--------

1	A–F (6)	Done
---	---------	------

2	A–D (4)	Done
---	---------	------

3	A–B (2)	Done
---	---------	------

4	A–C (5)	Done
---	---------	------

5	A–D (4)	Blocked
---	---------	---------

6	A–B (2)	Blocked
---	---------	---------

12 Memory Budget: Why 121 GB Can Fill Up

The DGX has 121 GB *total* for CPU + GPU. Here's where it goes during training:

- **Model params (bf16):** $1.4\text{B} \times 2 \text{ bytes} = 2.8 \text{ GB}$
- **Gradients (bf16):** 2.8 GB
- **AdamW optimizer states (fp32):** $m + v = 2 \times 1.4\text{B} \times 4 = 11.2 \text{ GB}$
- **Activations** (batch=128, L=256): $\approx 4 \text{ GB}$
- **Training data tensors:** $\approx 15 \text{ GB}$ (capped at 30 GB)
- **Overhead / CUDA context:** $\approx 5 \text{ GB}$

Total: $\approx 41 \text{ GB}$ for a typical training run. The data cap (30 GB) prevents the 121 GB limit from being reached.

The tokenization scripts were recently fixed — the original code stored token IDs as Python `list[int]` (28 bytes each) instead of tensor `int64` (8 bytes each). For 10 million steps, this alone wasted 72 GB, causing OOM before training even started. See `docs/reports/BUGFIX_TOKENIZE.md` and `INCIDENT_REPORT_TOKENIZE_OOM.md`.

13 Current Gaps (July 2026)

Bugs to Fix (0 GPU-hours)

1. Barrier formula in `lmc_barrier_scan.py` computes $\max(L) - L[0]$ instead of the correct Frankle definition $\max(L) - (L[0] + L[-1])/2$. Paper numbers were manually corrected.
2. Six fake LMC result files (`lr=2e-4/3e-4`) are byte-identical copies. Models exist but scans were never run.
3. `CLEANUP_MANIFEST.md` claims 4 files were deleted but they still exist on disk (`VISION.md`, `DIRECTION.md`, `THEORY.md`, `OPT_SECOND_MODEL.md`).

Experiments Needed

1. **GPT-Neo-1.3B replication** (highest priority). Status: scripts exist, results unknown.
2. **OPT upgrade to 11-point scans** (currently 3-point only for trajectory/Gaussian).
3. **ICLR Sprint Plan Phase 1** (≈ 9 GPU-hours): re-run `lr=2e-4/3e-4` scans, resolve upper-bound ambiguity, expand high-div medical seeds.

14 How to Be Productive as a Newcomer

1. **Read `CLAUDE.md` first.** It's the AI's instruction file — it tells you what the project is, what NOT to do, and the 10 iron rules.
2. **Read `ENGINEERING.md` §2.** The 23 bugs are better than any tutorial — each one teaches a non-obvious property of this machine or codebase.
3. **Bookmark the key data locations:** `experiments/phase0_ivn/results/` (122 JSONs), `paper.tex` (LaTeX source), `FIGURE_SPEC_V2.md` (figure specs).
4. **Start with a small experiment.** Train one model (`train_agent.py -domain code`), verify ΔW , run one LMC scan. This end-to-end path validates everything.
5. **Use `scripts/monitor.sh`** to check what's running and what's pending.
6. **When things break**, 90% of the time it's a known bug. Search `ENGINEERING.md` before debugging from scratch.
7. **For new experiments**, copy the pattern from `phase1_batch.sh` or `gptneo_pipeline.sh`. Follow the 10 iron rules rigorously.

15 Quick Reference: Key Commands

Task	Command
Train code model	<code>\$VENV -u scripts/train_agent.py -domain code -lr 1e-4</code>
Train medical model	<code>\$VENV -u scripts/train_agent.py -domain medical -lr 1e-4</code>
LMC scan (11-pt)	<code>\$VENV -u scripts/lmc_barrier_scan.py</code>
LMC scan (3-pt, fast)	<code>\$VENV -u scripts/lmc_3pt_scan.py -model-a DIR_A -model-b DIR_B</code>
Verify ΔW after training	<code>\$VENV -c "import torch; a=torch.load('base.pt'); b=torch.load('trained.pt'); ..."</code>
Monitor status	<code>bash scripts/monitor.sh</code>
Generate figures	<code>\$VENV scripts/generate_figures.py && \$VENV scripts/generate_fig4.py</code>
Compile paper	<code>cd docs/internal && pdflatex paper.tex && pdflatex paper.tex && cp paper.pdf ../../paper.pdf</code>
Sync from GitHub	<code>git pull origin main</code>
GPT-Neo pipeline	<code>bash scripts/gptneo_pipeline.sh</code>

16 Color Palette (for Figures)

Entity	Hex	Color
Pythia / Code domain	#2196F3	 Blue
Medical domain	#F44336	 Red
OPT	#FF5722	 Deep Orange
GPT-Neo / General domain	#4CAF50	 Green
Math domain	#FF9800	 Orange
Gaussian / Noise	#9E9E9E	 Gray
High divergence	#9C27B0	 Purple

This guide is written in L^AT_EX. Source: docs/reports/ONBOARDING_GUIDE.tex.

Recompile with: `pdflatex ONBOARDING_GUIDE.tex`.

Repository: <https://github.com/FauReam/AFP>